Tripare AI – React Native (Software Engineer)

Type: Production-Grade Mobile App (React Native + TypeScript)

Overview

Build a production-ready Dog Breed Explorer app. This assesses your ability to design resilient mobile systems that handle real-world constraints like unreliable networks and messy source data.

Framework Choice: React Native with TypeScript (Expo optional—use bare RN if you prefer)

Core User Stories

Breed Explorer: "I browse all 283 breeds smoothly, even without network, with smart caching and clear sync indicators."

Filtered Discovery: "I narrow down 283 breeds by group, size, coat, and temperament until I find the right fit."

Functional Requirements

1. Data Layer & Offline Architecture (Critical)

Dog API (v2), no key required: https://dogapi.dog/docs/api-v2

- `GET /api/v2/breeds` — 283 records, paginated at 48/page (6 pages to fetch and merge)
- `GET /api/v2/breeds/:id` — single breed, for detail refresh/enrichment
- `GET /api/v2/groups` — 9 breed groups, for filter labels

Offline-First Strategy:

- Local persistence (SQLite/AsyncStorage — your choice)
- App works fully offline after initial fetch
- Background sync when network returns
- Clear UI indicators for data freshness ("Last synced 2 hours ago")
- Handle partial failures (show cached data + error banner)
- Image caching with size limits — each breed carries up to 9 images with thumb/medium/large variants; decide which variant(s) to cache and when to upgrade

Performance:

- Virtualized list handling 283 records at 60fps — note these rows are heavier than a flat record: each carries nested traits, coat, origin, and image data, so the perf work here is about payload richness per row rather than sheer row count. Call this trade-off out explicitly in `DECISIONS.md`.
- Intelligent pagination/windowing while assembling the 6 API pages into one dataset
- Image caching with size limits

2. Breed List Screen

Features:

- Search: debounced search by breed name or `other_names`
- Filters (multi-select):
    - Breed group (Herding, Hound, Sporting, Terrier, Toy, Working, Non-Sporting, Miscellaneous, Foundation Stock Service)
    - Size band (derive your own buckets — e.g. small/medium/large/giant — from weight or height ranges)
    - Coat length (short/medium/long/wire)
    - Hypoallergenic (yes/no)
    - Trait threshold (e.g. good with children / good with dogs / good with strangers, above a chosen score)
- UI: grouped by breed group
- Actions: pull-to-refresh, tap to view details

Performance Target: Smooth 60fps scrolling across all 283 records, including list thumbnails

3. Breed Details Screen

Tabs: Overview | Traits | Gallery

- **Overview:** description, life span, weight/height ranges (male & female), origin (era/region/country), other names, recognizing kennel clubs
- **Traits:** the 11 numeric trait scores (energy, barking, drooling, grooming, shedding, trainability, good_with_dogs, exercise_minutes, apartment_friendly, good_with_children, good_with_strangers) plus temperament tags, rendered as a visual scale — not a raw table
- **Gallery:** swipeable image gallery; each image carries author/license/source attribution in the API response — surface it, don't drop it

Performance

- **Initial Load:** <3s to interactive (with cached data)
- **List Scroll:** 60fps sustained
- **Memory:** <150MB under normal use

Technical Architecture

Required Patterns

- **State Management:** Choose Zustand, Jotai, Redux Toolkit, or Context+Reducer. Justify choice in docs. Normalized cache for breeds/groups.
- **Data Persistence:** SQLite, WatermelonDB, or AsyncStorage. Document schema design.
- **API Layer:** React Query, SWR, or a custom hook with cache. Retry logic with exponential backoff. You'll need to fetch and merge 6 paginated pages into a single 283-breed dataset — handle that cleanly.
- **Navigation:** React Navigation or Expo Router. Type-safe navigation.
- **Error Handling:** Error boundaries around major features.

Code Quality Standards

- TypeScript strict mode enabled
- No `any` (use `unknown` with type guards)

Non-Functional Requirements

Performance

- Bundle Analysis: Document main bundle size
- Memory Profiling: Screenshot of memory usage during scroll

Deliverables

1. GitHub Repository Structure

tripare-dog-breeds/

├── docs/

│ ├── ARCHITECTURE.md # System design, data flow diagram

│ ├── DECISIONS.md # Technical choices + rationale

│ └── PERFORMANCE.md # Profiling results, bundle analysis

├── src/

│ └── __tests__/ # Test files

├── .env.example # All environment variables

├── README.md # See below

└── package.json

2. README.md Requirements

Must include:

Quick Start

npm install

cp .env.example .env

npm run ios # or npm run android

Should work in <3 minutes

Architecture Overview

- 2-3 paragraph summary
- Simple diagram (mermaid or image)
- Data flow: API → Cache → UI

Key Technical Decisions

- State management choice (why?)
- Database choice (why?)
- Offline sync strategy (how?)

Performance Report

- Bundle size breakdown
- Profiler screenshot (full 283-breed list render)
- Memory usage screenshot
- FPS during interactions

Screenshots (4-6 images)

- Breed list (light + dark mode)
- Filters applied
- Detail screen with traits and gallery
- Offline banner

3. Demo

Running App: Works on iOS/Android simulator/device

What we're NOT expecting:

- Pixel-perfect design (functional > beautiful)
- 100% test coverage
- Production deployment setup

Bonus Points (Optional)

- E2E Test: One Detox/Maestro test for critical flow
- Accessibility: Full screen reader support
- CI/CD: GitHub Actions for tests

Submission

- GitHub repo (public or private with access)
- README with all required sections
- App runs on first try
- Tests pass: `npm test`

Questions encouraged before starting—we prefer you clarify assumptions vs. guessing.